AI Agents Guide

Standing instructions for any AI coding agent building this product: what to read, what order to build in, the rules it must not break, and when a task counts as done. Save this as `AGENTS.md` (or `CLAUDE.md`) at the repo root, with the other five documents under `docs/`.

I could not find a published skill named "reverse engineer anything (rea)", so this guide is written to work with any agent workflow. If your skill expects specific file names or sections, tell me and I will reshape it.

1. Mission

Build an original, multi-tenant QR ordering platform for restaurants and canteens, to the PRD, on the architecture specified, using the design system. Reproduce the capability described in the reference case study, not its brand: no copied names, text, logos or imagery.

2. Source of truth

Question	Document	If documents conflict
What and why	`docs/01-prd.md`	PRD wins on scope
How it is built	`docs/02-architecture.md`	Architecture wins on technical design
How it looks	`docs/03-design-system.md`	Design System wins on UI
How it behaves	`docs/06-application-flow.md`	Flow wins on state and navigation
How it is verified	`docs/05-testing-guide.md`	Testing Guide wins on done-ness

If something is unspecified, choose the simplest option consistent with these documents, record it in `docs/decisions.md` (date, decision, reason), and continue. Stop and ask only for the items in section 9.

3. Non-negotiable rules

1. **Tenant isolation lives in the database.** Every table has RLS enabled with default deny. Never ship a table without policies and a policy test.

2. **Never trust the client for money.** Prices, taxes and totals are recomputed on the server with `computeBill` from `packages/core`.
3. **Order status changes only through the state machine** in `packages/core`, on the server, and each change writes an `order_events` row.
4. **The payment webhook is the source of truth.** Client-side success is a hint. Handlers are idempotent.
5. **Guests never get database credentials.** Guest traffic goes through route handlers that validate the QR token and session cookie.
6. **Secrets never enter the repo or the client bundle.** The service role key is server-only.
7. **UI uses design tokens and `components/ui` primitives.** No raw hex values, no one-off components that duplicate a primitive.
8. **Money is integer paise.** No floating point arithmetic on currency.
9. **Schema changes are migrations.** Never edit a migration that has been applied; add a new one.
10. **No new dependency without a reason** logged in `docs/decisions.md`. Prefer what is already in the stack.
11. **No fabricated data in production paths.** Seed data lives in `supabase/seed.sql` only.
12. **Original brand only.** Do not use the reference product's name, copy or assets anywhere.

4. Coding conventions

Topic	Convention
TypeScript	`strict: true`; no `any`; types derived from Zod schemas and generated DB types
Files	kebab-case files; PascalCase components; one component per file
Components	Server components by default; `"use client"` only where state or effects are needed
Data fetching	Server components or TanStack Query; no fetch in `useEffect`
Errors	Route handlers return `{ error: { code, message } }` with correct HTTP status; user-facing messages follow the microcopy rules

Topic	Convention
Validation	Zod at every boundary: request bodies, env vars, webhook payloads
Naming	DB snake_case; TS camelCase; status values exactly as in the Architecture state machine
Commits	Conventional Commits, one logical change each
Branches	`feat/<task-id>-short-name`; PR per task
Comments	Explain why, not what
Accessibility	Every interactive element keyboard reachable and labelled

5. Build order

Work strictly in this order; each task ends with its tests passing. Task IDs map to PRD requirement IDs.

#	Task	Covers	Done when
T01	Scaffold monorepo, lint, typecheck, CI, env validation	Arch 1, 15	CI green on an empty app
T02	Tokens, Tailwind config, `components/ui` primitives with a preview page	Design 2 to 6	Primitives render in light and dark
T03	Schema migrations: tenants, outlets, members, tables, menu	Arch 5	Migrations apply cleanly from zero
T04	RLS policies and pgTAP tests	Arch 4, 12	Cross-tenant read and write tests fail as expected
T05	Staff auth, onboarding (create tenant and outlet), role guard	D-08, D-09	Owner signs up and lands on an empty dashboard
T06	`packages/core`: Zod schemas, `computeBill`, state machine, with unit tests	Arch 6, 10	100% branch coverage on both
T07	Menu management UI and API, image upload	D-04	30-item menu built through the UI

#	Task	Covers	Done when
T08	Tables and QR generation, PDF export, regenerate	D-05	Printed QR opens the right table
T09	Guest menu: header, categories, search, filters, item sheet	G-01 to G-04, G-12	Lighthouse mobile performance 90+
T10	Cart, session, place order with idempotency	G-05 to G-07, G-09	Double-tap creates one order
T11	Schema for orders, order_items, order_events; realtime channels	Arch 5, 8	Events received by a test subscriber
T12	Guest order tracking	G-08	Status change visible in under 2 s
T13	Kitchen screen	K-01 to K-06	Works through a simulated network drop
T14	Live orders board, table tiles, order drawer, accept and reject	D-01 to D-03	Full dine-in run-through passes
T15	Payments: create, checkout, verify, webhook, cash, refund	G-10, D-06, D-07	Test-mode payment reconciles; duplicate webhook is harmless
T16	Bill, GST invoice PDF, close table	D-06, G-11	Invoice totals equal `computeBill` output
T17	Canteen mode: tokens, prepaid rule, canteen panel, pickup display	G-14, C-01 to C-04	Full canteen run-through passes
T18	Service requests: call waiter, request bill	G-15	Request appears on the dashboard in realtime
T19	Analytics rollup job and dashboard	D-10	Numbers match a hand-computed seed dataset
T20	Platform admin: tenants, suspend	A-01	Suspended tenant's QR shows a closed page
T21	Hardening: rate limits, Sentry, a11y pass, load test, PWA polish	NFRs	Release gates in the Testing Guide pass

Phase 2 backlog, not to be started without instruction: upsells, inventory, printing, split bill, coupons, feedback, languages, POS adapters.

6. Per-task workflow

1. Read the task row, the linked PRD requirements and the relevant Architecture and Flow sections.
2. Write a short plan: files to add or change, schema impact, tests to add.
3. Write or update tests first for `packages/core` logic and RLS; alongside for UI.
4. Implement the smallest change that satisfies the acceptance criteria.
5. Run `pnpm lint && pnpm typecheck && pnpm test`, then the relevant Playwright spec.
6. Self-review against section 3 and the definition of done.
7. Open a PR with: what changed, requirement IDs covered, screenshots for UI, test evidence, any decisions logged.

7. Definition of done

- Acceptance criteria for the mapped requirement IDs are demonstrably met.
- Lint, typecheck, unit, policy and relevant end-to-end tests pass in CI.
- New tables have RLS policies and policy tests.
- New endpoints have Zod validation, auth checks, rate limiting where guest-facing, and tests for the unauthorised path.
- UI matches tokens, has loading, empty and error states, and passes an automated accessibility scan.
- No secrets, no console errors, no TODOs without a linked issue.
- Docs updated if behaviour or schema changed.

8. Commands

Purpose	Command
Install	`pnpm install`
Local database	`supabase start` then `supabase db reset`
Dev server	`pnpm dev`
Lint and types	`pnpm lint` and `pnpm typecheck`
Unit tests	`pnpm test`
Policy tests	`supabase test db`

Purpose	Command
End-to-end	`pnpm e2e`
Generate DB types	`supabase gen types typescript --local > packages/core/src/db-types.ts`

9. Stop and ask a human before

- Deleting data, dropping tables or columns, or rewriting applied migrations.
- Touching production environments, live payment keys or DNS.
- Changing the pricing, tax or state-machine rules.
- Adding a paid service or a dependency with a restrictive licence.
- Anything that weakens RLS, auth or webhook verification, even temporarily.
- Any step that needs a credential you were not given. Never invent one.

10. Suggested agent roles

Useful if you run several agents; one agent can play all roles in sequence.

Role	Owns	Reads first
Planner	Breaks a task into steps, checks dependencies	PRD, this guide
Data agent	Migrations, RLS, policy tests, seed	Architecture 4, 5, 12
API agent	Route handlers, `packages/core`, payments, realtime	Architecture 6 to 10
UI agent	Components and screens	Design System, Application Flow
Test agent	Unit, end-to-end, load, accessibility	Testing Guide
Reviewer	Fresh-context review of each PR against sections 3 and 7	Everything

The reviewer should not be the agent that wrote the code, and should start without the author's context.

11. Kick-off prompt

```
You are building the product specified in /docs. Read AGENTS.md fully, then
docs/01-prd.md and docs/02-architecture.md. Do not write code yet.
1. List any contradictions or gaps you find, with a proposed resolution each.
2. Confirm the build order in AGENTS.md section 5 or propose changes.
3. Then start task T01 and follow the per-task workflow in section 6.
Rules in AGENTS.md section 3 are absolute. Ask before anything in section 9.
```

12. Task prompt template

```
Task: <T-id and name>
Requirements: <PRD IDs>
Read: <doc sections>
Acceptance: <the "Done when" line plus requirement text>
Constraints: AGENTS.md sections 3 and 4.
Deliver: plan, code, tests, PR description with evidence.
```

13. AI features inside the product (later)

None are needed for the MVP. Candidates once real order data exists: menu description drafting for owners, upsell suggestions from co-purchase data, demand forecasting for prep and staffing, and a natural-language analytics query box. Each must be optional, must show its source data, and must never alter prices or orders without staff confirmation.